

Laboratorio di Sistemi Operativi

A.A. 2023-24

FIFO(ni)

francesco.fuligni@studio.unibo.it

Paolo Casula - 0001070254

Francesco Maria Fuligni - 0001068987

Emiliano Serranti - 0001077422

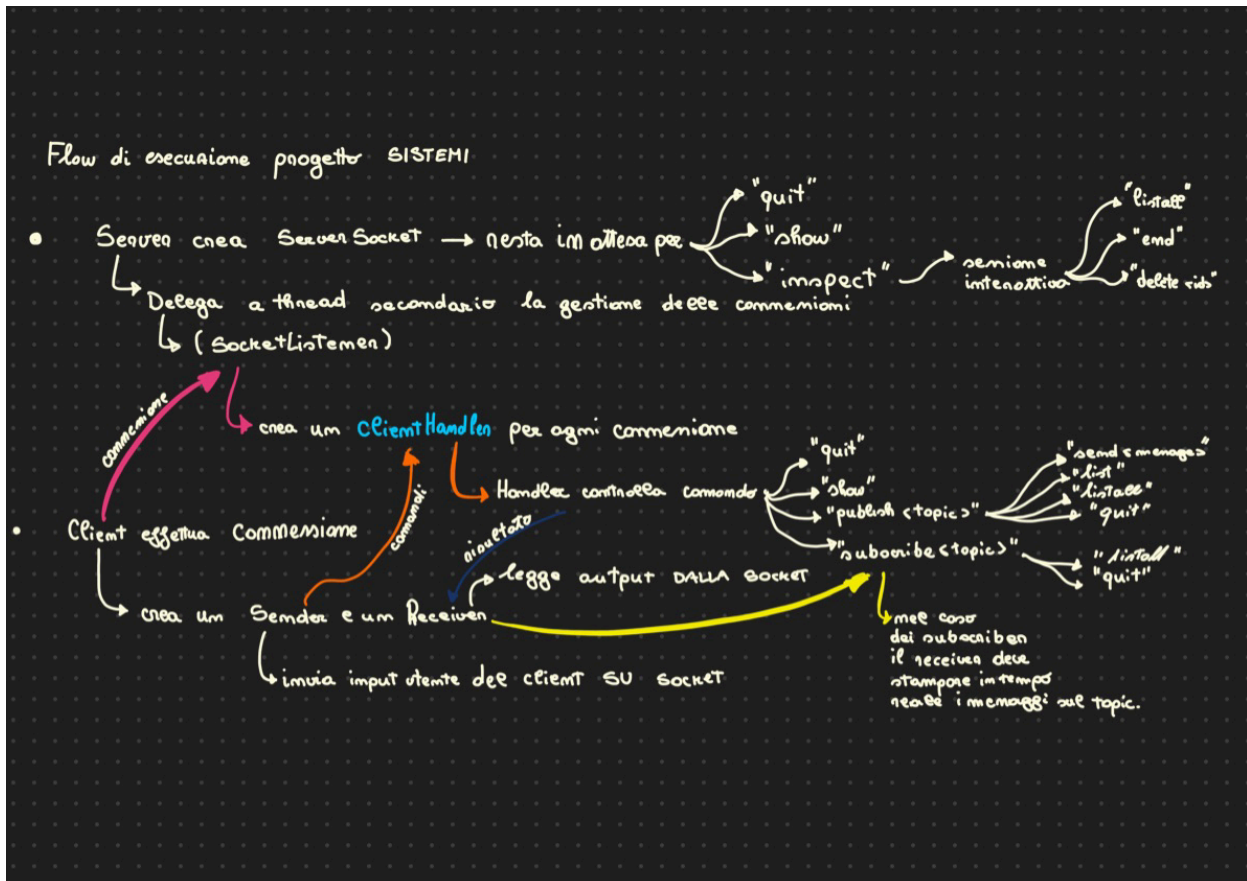
Roberto Zanolli - 0001070505

Sommario

1. Descrizione del progetto.....	2
(a) Architettura generale.....	2
(b) Descrizione dettagliata delle singole componenti.....	3
Lato client.....	3
Lato server.....	3
(c) Suddivisione del lavoro tra i membri del gruppo.....	7
2. Descrizione e discussione del processo di implementazione.....	8
(a) Descrizione dei problemi e degli ostacoli incontrati.....	8
Modello client-server.....	8
Concorrenza.....	8
(b) Descrizione degli strumenti utilizzati per l'organizzazione.....	9
3. Requisiti e istruzioni per compilare e usare le applicazioni.....	10
Avvio server.....	11
Avvio client.....	11
Publisher.....	13
Subscriber.....	15
Sessione interattiva e chiusura del Server.....	17

1. Descrizione del progetto

(a) Architettura generale



Il progetto implementa un servizio publish/subscribe, dove multipli client possono iscriversi a determinati topic, e inviare o ricevere i messaggi inviati su di essi, con diverse funzionalità a seconda del ruolo di publisher o subscriber.

Un Client, una volta connesso al Server, interagisce con esso tramite 2 thread: il Sender e il Receiver.

Il primo gestisce la lettura dell'input dell'utente e invia i comandi al Server mentre il secondo gestisce la ricezione dei messaggi da parte del Server e li visualizza sulla console del Client.

Il server controlla le connessioni in ingresso e crea un ClientHandler per ciascun Client.

I ClientHandler sono responsabili dell'elaborazione dei comandi del Client.

Il TopicsHandler ha un elenco di tutti i Topic e interagisce con il ClientHandler per gestire le iscrizioni e le pubblicazioni dei Client.

(b) Descrizione dettagliata delle singole componenti

Lato client

Client
+main(String[]) : void

Client permette all'utente di interagire con il server, avvia una connessione con esso e crea un thread per inviare messaggi (Sender) e uno per riceverli (Receiver).

L'utilizzo di thread ausiliari per l'invio e la ricezione di messaggi permette di ottimizzare la gestione dell'applicazione.

Con il metodo join, attende il termine dei thread Sender e Receiver creati prima di terminare l'esecuzione.

Sender
-s : Socket
+Sender(Socket) +run() : void

La classe **Sender** legge l'input da tastiera mediante uno scanner e lo invia al server tramite socket, utilizzando un oggetto PrintWriter.

Receiver
-s : Socket -sender : Thread
+Receiver(Socket, Thread) +run() : void

La classe **Receiver** riceve messaggi dal server tramite socket, rimanendo in continuo ascolto sulla socket, e stampa i messaggi ricevuti sulla console del client.

Lato server

Server
+topics : TopicsHandler
+main(String[]) : void

Server accetta e gestisce le connessioni in ingresso dai client usando un `ServerSocket` in ascolto. Mantiene la risorsa comune `topics`, ovvero l'insieme di tutti i topic creati con i metodi per accedervi e modificarli.

Il `Server` può gestire più client contemporaneamente, lasciando il compito di gestire le connessioni al `SocketListener`, il thread deputato a questa funzione.

Inoltre in `Server` si gestiscono i comandi `quit`, `show` ed `inspect`. Quest'ultimo avvia la sessione interattiva invocando il metodo dedicato nella classe `Topic`.

SocketListener
- server : <code>ServerSocket</code> - children : <code>List<Thread></code>
+ <code>SocketListener(ServerSocket)</code> + <code>run()</code> : void

La classe **SocketListener** rappresenta un listener lato server che accetta le connessioni da parte dei client in ciclo continuo finché il thread principale non viene interrotto.

Per la gestione di un nuovo client viene avviato un nuovo thread con un'istanza di `ClientHandler` associata e viene aggiunto alla lista `children`, in modo da tenere traccia dei client in esecuzione.

Quando il thread principale viene interrotto viene chiuso il `ServerSocket` ed è inviato un segnale di interruzione a ciascun thread scorrendo la lista `children`.

È necessario l'utilizzo di un timeout essendo `accept()` un'operazione bloccante.

Qualora venga raggiunto il timeout viene sollevata una `SocketTimeoutException`, dopo la quale viene controllato nuovamente lo stato del thread nella condizione del `while()`.

ClientHandler
-socket : <code>Socket</code> -topicName : <code>String</code> -clientID : <code>String</code> -role : <code>Role</code>
+ <code>ClientHandler(Socket)</code> + <code>getSocket()</code> : <code>Socket</code> + <code>run()</code> : void

<<enumeration>> Role -publisher -subscriber -undefined

La classe **ClientHandler** gestisce la comunicazione tra il server e un singolo client.

Ad ogni connessione da parte di un client viene creata un'istanza dedicata di ClientHandler che opera in un thread separato e gestisce le richieste del client, analizzando la stringa in input e invocando i metodi corrispondenti o restituendo messaggi di errore.

Il ruolo di un client è inizialmente *undefined*. Assumerà in seguito il valore *publisher* o *subscriber* quando si registrerà ad un topic rispettivamente con il comando publish o subscribe. Tenere traccia del ruolo è necessario per filtrare i comandi disponibili e restituire i messaggi di errore a seconda delle casistiche.

Il campo `topicName` tiene traccia del topic su cui il client si è registrato, permettendo, alla ricezione di un comando, di invocare il metodo corrispondente accedendo direttamente al topic corretto. L'accesso è diretto dato che i topic sono gestiti tramite una HashMap in TopicsHandler che associa il nome del topic all'oggetto topic.

Il campo `clientID` associa al client un identificativo realizzato tramite una marcatura dell'istante temporale in cui è avvenuta la connessione al server, il che lo rende univoco. L'id è fondamentale per poter identificare i singoli client e associare ad ognuno i messaggi da lui inviati su un topic.

Message
-id : String -text : String -date : LocalDateTime
+Message(String, int) +getID() : String -printDate() : String +toString() : String

La classe **Message** rappresenta un singolo messaggio.

Ogni messaggio possiede un identificativo univoco, generato da un contatore a livello del singolo topic in maniera incrementale. Dunque, il primo messaggio sul topic avrà un id del tipo "msg_1", il secondo "msg_2" e così via.

Si noti che l'id è univoco a livello di topic, ma non a livello dell'insieme di topic: il primo messaggio su un topic A avrà lo stesso id del primo messaggio sul topic B ("msg_1"). Questo non rappresenta un problema per il funzionamento dell'applicazione in quanto per accedere ai messaggi è sempre necessario specificare prima il topic di riferimento.

Ogni messaggio ha anche un campo contenente la data e l'orario di ricezione. Questi attributi sono inizializzati al momento di creazione del messaggio, che corrisponde effettivamente al

momento in cui viene ricevuto, dal momento che l'oggetto Message viene istanziato nel metodo send() all'interno della classe Topic.

Come si può notare eseguendo l'applicazione, si è deciso di adottare una rappresentazione dell'orario di ricezione nel formato HH:mm:ss.SSSSSS in quanto permette di individuare con maggiore precisione l'ordine di ricezione dei messaggi, utile anche per osservare come avviene il context switch tra i processi al momento del notifyAll.

Topic
-messages : Map<String, List<Message>> -name : String -subscribers : Set<ClientHandler> -IDcount : int -listCount : int
+Topic(String) -getClientMessages(String) : List<Message> -getAllMessages() : List<Message> -findMessage(String) : Message -notifySubscribers(Message) : void -deleteMessage(String) : boolean -startList() : void -endList() : void +send(String, String) : void +subscribe(ClientHandler) : void +unsubscribe(ClientHandler) : void +list(String) : String +listAll : String +interactiveSession(Scanner) : void +toString : String

La classe **Topic** è centrale nel corretto funzionamento dell'applicazione e dei meccanismi per la gestione dei processi concorrenti.

Ogni topic è caratterizzato da un nome (univoco, essendo i topic memorizzati in una HashMap, vedasi la classe TopicsHandler), un insieme di client iscritti al topic, un insieme di messaggi, un contatore progressivo per gli id dei messaggi e un altro contatore che tiene traccia dei client in lettura sul topic, per la gestione della concorrenza.

Topic memorizza i messaggi in una mappa avente come chiave l'ID del client che ha inviato il messaggio e come valore una lista di messaggi associati a quel client. In questo modo, è possibile restituire facilmente la lista dei messaggi inviati da un singolo client.

La classe contiene i metodi send() per inviare messaggi e notifySubscriber() per notificare automaticamente i subscriber di un nuovo messaggio su un determinato topic.

Contiene i metodi list() e listAll() invocati dal ClientHandler quando richiesti da un utente. Essi utilizzano rispettivamente getClientMessages() e getAllMessages(), metodi privati presenti nella classe, per formattare l'output da restituire al client.

La gestione dei client iscritti al topic avviene tramite i metodi `subscribe()` e `unsubscribe()` che consentono di aggiungere (al momento del `subscribe`) e rimuovere (al momento del `quit`) i subscriber associati al topic.

Il metodo `interactiveSession()` può essere invocato solamente dal server tramite il comando `inspect` per avviare la sessione interattiva e utilizzare i comandi specifici `listAll`, `delete` ed `end`.

La classe `Topic` gestisce la maggior parte dei problemi di concorrenza dell'applicazione.

In primo luogo, le operazioni di lettura dei messaggi tramite `list()` e `listAll()` non devono interferire con l'operazione di scrittura `send()`, mentre non devono essere ammessi `send()` contemporanei. A tale scopo, il metodo `send()` è definito come `synchronized` così come i metodi `startList()` ed `endList()`, invocati all'inizio e al termine dei metodi `list()` e `listAll()`, che incrementano e decrementano il contatore `listCount` dei client in lettura sul sistema. Tale modello ricalca la soluzione al problema dei lettori e scrittori presentata durante il corso di Sistemi Operativi, con un `notifyAll()` al termine dell'esecuzione dei lettori (quando il contatore è `= 0`) e un `wait()` nell'operazione di scrittura (finché il contatore è `> 0`).

Il metodo privato `notifySubscribers()` non è sincronizzato dal momento che viene eseguito all'interno del metodo `send()`, già `synchronized`.

`InteractiveSession()` è definito `synchronized` in modo che i comandi `send`, `list` e `listAll` dei publisher e subscriber rimangano in attesa durante l'ispezione di un `Topic` da parte del Server. Analogamente, per poter analizzare il topic, il server deve attendere il termine delle operazioni dei client sul topic stesso per evitare interferenze.

Infine, i metodi `subscribe()` e `unsubscribe()` non sono sincronizzati in quanto, se lo fossero, renderebbero impossibile l'iscrizione al topic o il `quit` durante la sessione interattiva del server, la lettura o l'invio di messaggi. L'assenza di interferenze sul Set di subscribers è garantita dall'utilizzo della classe thread-safe `ConcurrentHashMap`.

TopicsHandler
-topics : Map<String, Topic> -readCount : int
+TopicsHandler() -startRead() : void -endRead() : void +get(String) : Topic +putIfAbsent(String) : void +contains(String) : boolean +show() : String

La classe **TopicsHandler** si occupa di gestire l'insieme dei topic creati e contiene vari metodi per la loro manipolazione. Per memorizzarli, sfrutta una `HashMap` in cui la chiave è il nome del topic mentre il valore è l'oggetto `Topic` corrispondente. L'utilizzo di `HashMap` è conveniente in quanto rende le operazioni di accesso al singolo topic più efficienti.

`TopicsHandler` dispone anche di un contatore per tracciare il numero di thread in lettura sull'insieme dei topic, utile alla gestione della concorrenza.

La classe permette, con `putIfAbsent()`, di aggiungere un nuovo topic (se non già esistente), di controllare, con `contains()`, se un topic è già presente nel dizionario e di mostrare, con `show()`, tutti i topic creati dai client.

Il metodo `putIfAbsent()` è `synchronized` per garantire la mutua esclusione tra due thread che stanno creando un nuovo topic. Altrimenti, potrebbe accadere che a causa di un context switch avvenuto prima del termine della creazione di un topic, alcuni dati vengano sovrascritti.

I metodi `startRead()` ed `endRead()` permettono di gestire la concorrenza, risolvendo un problema analogo a quello di `send()`, `list()` e `listAll()` nella classe `Topic`. Essi sono utilizzati all'inizio e alla fine dei metodi `contains()` e `show()`, permettendo la lettura contemporanea di più client sulla risorsa. In questo modo, il metodo `putIfAbsent` verrà eseguito solamente se non ci sono altre operazioni di lettura (o scrittura) in esecuzione.

(c) Suddivisione del lavoro tra i membri del gruppo

La suddivisione del lavoro è stata gestita in maniera flessibile e dinamica, in base alle esigenze e alle abilità dei membri del gruppo. Di fatto, tutti i componenti del gruppo hanno preso parte ad ogni fase dello sviluppo, contribuendo in maniera collaborativa.

In linea generale, il lavoro è stato suddiviso in fasi successive di implementazione delle funzionalità dell'applicazione, in cui ogni componente del gruppo:

- Roberto Zanolli si è occupato in primo luogo della gestione della comunicazione tra Client e Server.
- Emiliano Serranti e Paolo Casula hanno contribuito a progettare la struttura della gestione delle richieste e dell'implementazione dei comandi per i client e il server.
- Francesco Maria Fuligni si è occupato della struttura per la gestione e la memorizzazione dei topic e dei messaggi e dei vari aggiornamenti necessari alla struttura del codice per migliorare chiarezza ed efficienza.
- Roberto Zanolli si è occupato della gestione dei comandi lato Server.
- La parte di concorrenza è stata gestita in maniera collaborativa, lavorando collettivamente e attivamente per comprendere il funzionamento dell'applicazione e i principali punti critici per potenziali interferenze.

2. Descrizione e discussione del processo di implementazione

(a) Descrizione dei problemi e degli ostacoli incontrati

Modello client-server

Il primo problema da risolvere è stata l'implementazione della comunicazione tra client e server. Studiando le architetture viste durante il Laboratorio di Sistemi Operativi, è stata implementata una struttura in cui più client e un singolo server sfruttano classi ausiliarie per scambiarsi messaggi su una socket, come spiegato in precedenza nella descrizione delle componenti. Così facendo è stata semplificata la gestione delle richieste del client e delle risposte del server.

Concorrenza

Successivamente, è stato necessario comprendere come gestire i comandi provenienti dal Client al Server. La prima implementazione vedeva una gestione completa dei comandi all'interno della classe ClientHandler, sfruttando il fatto che l'applicazione istanzia un oggetto ClientHandler per ogni Client presente. Tuttavia, il codice prodotto risultava troppo complesso e poco efficiente in generale e dunque sono state adottate alcune correzioni:

- È stata creata una classe TopicsHandler contenente l'insieme di tutti i topic creati, con i vari metodi per l'accesso al singolo topic e l'implementazione dei comandi lato server. Questa risorsa è istanziata nel Server ed è condivisa ed accessibile a tutti i client.
- La classe Topic è divenuta centrale per il corretto funzionamento dell'applicazione, in quanto in essa sono stati implementati i metodi invocati dal client tramite i comandi.

Questo tipo di gestione si è dimostrata particolarmente efficace e funzionale per la gestione della concorrenza, permettendo di sincronizzare direttamente i metodi della classe Topic e TopicsHandler nei casi di possibili interferenze.

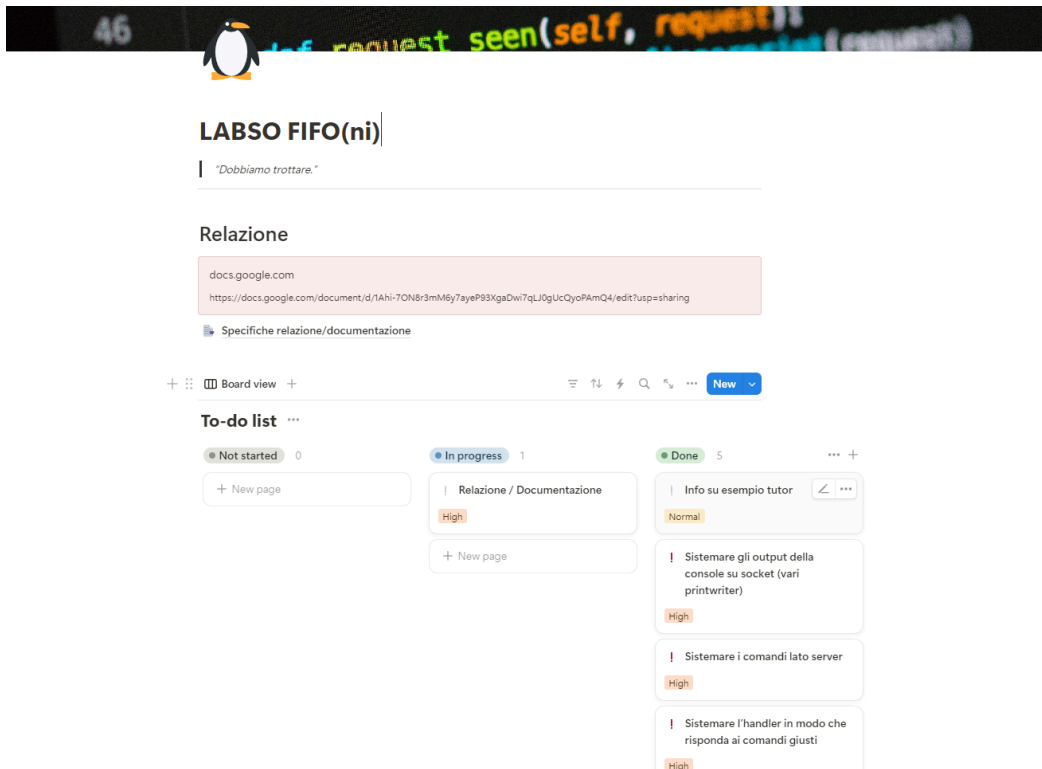
In particolare, le principali interferenze notate sono riconducibili al problema dei "Lettori e Scrittori" osservato durante le lezioni del corso di Sistemi Operativi, nel quale la lettura contemporanea di più thread deve essere garantita, ma la scrittura deve essere esclusiva. Conseguentemente, la concorrenza tra le operazioni di lettura e scrittura è stata gestita tramite un contatore che tiene traccia del numero di thread in lettura sulla risorsa condivisa di tipo TopicsHandler o sul singolo Topic, a seconda della casistica, e sincronizzando i metodi di scrittura e di incremento e decremento della variabile contatore.

Infine, un ultimo problema è stato incontrato nel gestire processi concorrenti in accesso al Set di subscribers in Topic. Inizialmente, i metodi unsubscribe e subscribe erano stati realizzati synchronized per evitare interferenze in scrittura dei dati, ma ciò impediva l'iscrizione ad un topic o il quit durante la sessione interattiva del server o durante l'esecuzione di altri metodi synchronized (send, startList, endList). Per evitarlo, si è deciso di sfruttare un lock a livello dell'oggetto Set e, per implementarlo, è stata sfruttata direttamente la classe thread-safe ConcurrentHashMap di Java.util.concurrent. Sfruttando il metodo newKeySet(), si ottiene un

insieme thread-safe. Questo ha permesso di evitare la scrittura di una classe privata aggiuntiva con metodi sincronizzati per richiamare le operazioni sul Set e dunque una maggiore chiarezza nel codice.

(b) Descrizione degli strumenti utilizzati per l'organizzazione

- Per sviluppare il progetto è stato utilizzato il software Visual Studio Code, insieme con GitHub Desktop per condividere il codice, da parte di tutti i componenti del gruppo. Ogni membro disponeva di un proprio branch su cui svolgere il lavoro, e al termine di ogni fase di lavoro, raggiunta un'implementazione funzionante e soddisfacente, si realizzava un commit sul main branch. Si specifica che il codice è stato realizzato in gran parte in sessioni collettive a cui tutti i membri hanno preso parte, ma le modifiche venivano realizzate solamente su un computer per facilitare il processo. Inoltre, a seconda delle necessità sono stati creati branch temporanei per gestire conflitti nei commit e per sperimentare soluzioni alternative.
- Per comunicare tra i membri del gruppo è stato usato prevalentemente un gruppo Whatsapp specifico per il progetto, mentre per le sessioni di lavoro collettivo da remoto sono stati utilizzati strumenti per videochiamate di gruppo, in particolare Google Meet e Discord.
- Per l'organizzazione generale del progetto, ossia tenere traccia del lavoro svolto e rimasto da svolgere e come mezzo di condivisione di risorse utili è stato sfruttato un Notion condiviso. In esso era presente una tabella con le task da svolgere, divise per stato di completamento, oltre ad una sezione di dispensa di teoria realizzata dai componenti del gruppo integrando le nozioni del corso di Sistemi Operativi.

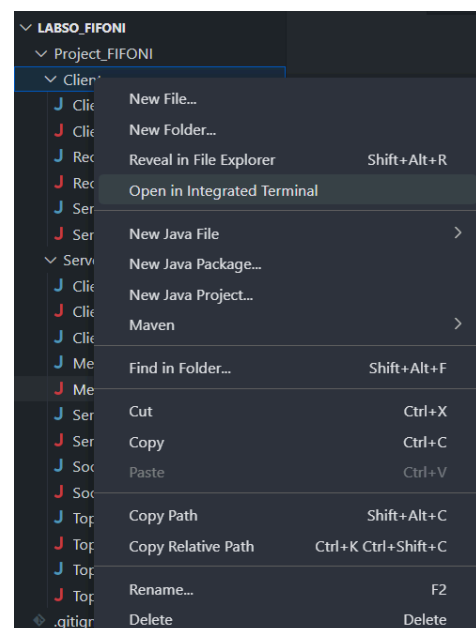


3. Requisiti e istruzioni per compilare e usare le applicazioni

Dopo aver aperto il progetto nell'editor di codice scelto, per avviare l'applicazione sarà necessario compilare tutti i file presenti all'interno del progetto.

In VSC è consigliato effettuare un hover con il mouse sul file explorer, in particolare sulle cartelle Client e Server, effettuare un right-click per ognuna di esse e cliccare successivamente "Open in integrated terminal". In questo modo verranno aperti due terminali, uno per il server e uno per il client. Nel caso in cui si volessero simulare più client connessi al server, è sufficiente aprire altri terminali, tanti quanti i client che si vogliono connettere.

Il processo analogo è realizzabile tramite il terminale integrato nel sistema operativo, modificando la directory selezionata in Client e Server.



Una volta aperte le due “Cartelle” nei terminali integrati, basterà inserire il comando: “javac *.java” in entrambi i terminali.

```
javac *.java
```

In questo modo verranno automaticamente compilati tutti i file delle due cartelle. Ora è possibile procedere con l’esecuzione delle applicazioni.

Avvio server

Una volta compilati tutti i file del progetto si procede avviando il lato Server con la linea di comando: “java Server <numero porta>”.

A scopo esemplificativo è stata scelta la porta 9000 e verrà utilizzata per tutti gli esempi; si nota che è possibile scegliere arbitrariamente una porta non già in uso da altri servizi.

```
java Server 9000
```

Nel terminale verrà visualizzata la seguente risposta:

```
Comandi server:
> show
> inspect <topic>
> quit
In attesa di connessioni...
█
```

Nel caso di connessione accettata correttamente il server visualizzerà il messaggio:

```
Client connesso.
Thread Thread[#24,Thread-3,5,main] in ascolto...
```

In questo caso varieranno i numeri che seguono il carattere “#” e i numeri che seguono “Thread-”.

Avvio client

A questo punto ci saranno due casi:

1. **Un singolo computer.** Si potrà effettuare la richiesta di connessione con la seguente linea di comando: “java Client localhost 9000”. Di nuovo, 9000 è il numero di porta ed è stato scelto per comodità. Ovviamente deve corrispondere con il numero di porta inserito all’avvio del Server, ma non deve per forza essere 9000.

```
java Client localhost 9000
```

2. **Più computer connessi alla stessa rete.** Bisognerà effettuare la richiesta di connessione con la seguente linea di comando: “java Client <indirizzo IP> 9000”.

Attenzione, al posto di “<indirizzo IP>” bisognerà inserire l’indirizzo IP del computer che sta svolgendo il ruolo di Server.

```
java Client 172.20.12.10 9000
```

Se la connessione è andata a buon fine, verrà visualizzato il messaggio di connessione con l'elenco dei comandi disponibili per il client:

```
Connesso al server.  
Comandi client:  
  > publish <topic>  
  > subscribe <topic>  
  > show  
  > quit  
█
```

Da questo momento in poi sarà possibile utilizzare i vari comandi, secondo il funzionamento precisato nelle specifiche del progetto.

Al contrario, nel caso in cui sia stato inserito un comando errato alla richiesta di connessione, verrà visualizzato un messaggio di errore specifico a seconda della casistica:

1. Errore in inserimento "Client".

```
java Clinet localhost 9000
```

```
Error: Could not find or load main class Clinet  
Caused by: java.lang.ClassNotFoundException: Clinet
```

2. Errore in inserimento "localhost".

```
java Client localhst 9000
```

```
CLIENT - IOException catturata: java.net.UnknownHostException: localhst  
java.net.UnknownHostException: localhst  
  at java.base/sun.nio.ch.NioSocketImpl.connect(NioSocketImpl.java:567)  
  at java.base/java.net.SocksSocketImpl.connect(SocksSocketImpl.java:327)  
  at java.base/java.net.Socket.connect(Socket.java:751)  
  at java.base/java.net.Socket.connect(Socket.java:686)  
  at java.base/java.net.Socket.<init>(Socket.java:555)  
  at java.base/java.net.Socket.<init>(Socket.java:324)  
  at Client.main(Client.java:34)
```

3. Errore in inserimento numero di porta.

```
java Client localhost 9001
```

```
CLIENT - IOException catturata: java.net.ConnectException: Connection refused: connect  
java.net.ConnectException: Connection refused: connect  
  at java.base/sun.nio.ch.Net.connect0(Native Method)  
  at java.base/sun.nio.ch.Net.connect(Net.java:589)  
  at java.base/sun.nio.ch.Net.connect(Net.java:578)  
  at java.base/sun.nio.ch.NioSocketImpl.connect(NioSocketImpl.java:583)  
  at java.base/java.net.SocksSocketImpl.connect(SocksSocketImpl.java:327)  
  at java.base/java.net.Socket.connect(Socket.java:751)  
  at java.base/java.net.Socket.connect(Socket.java:686)  
  at java.base/java.net.Socket.<init>(Socket.java:555)  
  at java.base/java.net.Socket.<init>(Socket.java:324)  
  at Client.main(Client.java:34)
```

4. Mancanza di uno degli argomenti necessari all'esecuzione:

```
java Client 9001
```

```
Utilizzo:  
> java Client <host> <porta>
```

Publisher

Un client può diventare “publisher” con il comando “publish <Stringa>”. Al posto di <Stringa> bisogna inserire una qualsiasi stringa non vuota.

In caso di Stringa vuota invece verrà visualizzato il seguente messaggio:

```
publish  
ERRORE: nessun topic specificato.  
□
```

Se il comando inserito è corretto, verrà visualizzato il seguente messaggio:

```
publish Notizie  
Registrato come publisher sul topic 'Notizie'.  
Comandi publisher:  
  > send <message>  
  > list  
  > listall  
  > quit
```

Ora sarà possibile mandare i messaggi nel Topic creato/selezionato con il comando “send <message>” inserendo il messaggio desiderato al posto di <message>. Non sono ammessi messaggi vuoti. In caso di stringa vuota verrà visualizzato l'errore:

```
send  
ERRORE: nessun messaggio specificato.  
□
```

In caso di inserimento corretto, invece:

```
send L'italia non riesce a superare i francesi di Deschamps  
Messaggio inviato sul topic 'Notizie'.
```

Sarà anche possibile effettuare il list e il listall:

Listall mostrerà tutti i messaggi inseriti nel topic (da tutti i publisher)

```
listall
Tutti i messaggi sul topic 'Notizie':
- ID: msg_5
  Testo: 'Clamorosa vittoria di Verstappen mette un'ipoteca sul campionato'
  Data: 21/11/2024 - 15:58:57.76364
- ID: msg_1
  Testo: 'L'italia non riesce a superare i francesi di Deschamps'
  Data: 21/11/2024 - 15:52:43.75897
- ID: msg_2
  Testo: 'Sinner vince le finali ATP di Torino'
  Data: 21/11/2024 - 15:55:21.42771
- ID: msg_3
  Testo: 'Juric viene esonerato dalla Roma, la societ? cerca un allenatore per sostituire l'allenatore croato'
  Data: 21/11/2024 - 15:56:10.92575
- ID: msg_4
  Testo: 'La Roma sceglie Ranieri come nuovo allenatore'
  Data: 21/11/2024 - 15:56:44.12476
```

List mostrerà solo quelli inviati dal publisher che sta richiedendo il list:

```
list
Messaggi inviati dal client 'clt_1732199394282' sul topic 'Notizie':
- ID: msg_1
  Testo: 'L'italia non riesce a superare i francesi di Deschamps'
  Data: 21/11/2024 - 15:52:43.75897
- ID: msg_2
  Testo: 'Sinner vince le finali ATP di Torino'
  Data: 21/11/2024 - 15:55:21.42771
- ID: msg_3
  Testo: 'Juric viene esonerato dalla Roma, la societ? cerca un allenatore p
  Data: 21/11/2024 - 15:56:10.92575
- ID: msg_4
  Testo: 'La Roma sceglie Ranieri come nuovo allenatore'
  Data: 21/11/2024 - 15:56:44.12476
```

Infine potrà effettuare il quit:

```
quit
Sender terminato.
Connessione chiusa.
Receiver terminato.
Socket chiusa.
```

Dopo ogni comando inviato dal Client, verrà visualizzata la relativa richiesta sul Server. Tale meccanismo si è rivelato particolarmente utile ai fini di debugging, ma abbiamo ritenuto fosse utile lasciarlo visibile a scopo esemplificativo del funzionamento dell'applicazione. Ecco un esempio di un insieme di richieste:

```
Richiesta: show
Richiesta: publish Notizie
Richiesta: send
Richiesta: publish
Richiesta: send L'italia non riesce a superare i francesi di Deschamps
Richiesta: list
Richiesta: listall
Richiesta: send Sinner vince le finali ATP di Torino
Richiesta: list
Richiesta: listall
Richiesta: send Juric viene esonerato dalla Roma, la societ? cerca un allenatore per sostituire l'allenatore croato
Richiesta: send La Roma sceglie Ranieri come nuovo allenatore
Richiesta: list
Richiesta: listall
Richiesta: show
Richiesta: publish Notizie
Richiesta: send Clamorosa vittoria di Verstappen mette un'ipoteca sul campionato
Richiesta: listall
Richiesta: list
```

In particolare, dopo il quit verrà visualizzato:

```
Richiesta: quit
Connessione chiusa.
```

In caso di chiusura forzata del client, dunque senza l'utilizzo corretto del “quit”, il server visualizzerà:

```
CLIENTHANDLER - NoSuchElementException catturata: java.util.NoSuchElementException: No line found
java.util.NoSuchElementException: No line found
    at java.base/java.util.Scanner.nextLine(Scanner.java:1660)
    at ClientHandler.run(ClientHandler.java:69)
    at java.base/java.lang.Thread.run(Thread.java:1583)
```

Subscriber

Per decidere a quale topic effettuare l'iscrizione si può usare il comando “show”:

```
show
Tutti i topic:
- Notizie
```

Con il comando “subscribe <topic>” sarà possibile effettuare l'iscrizione al <topic>. Al posto di <topic> bisogna inserire una stringa non vuota, corrispondente al topic desiderato. Nel caso di stringa vuota verrà visualizzato il seguente messaggio:

```
subscribe
ERRORE: nessun topic specificato.
```

Nel caso invece di iscrizione ad un topic inesistente, non ci saranno errori, in quanto il topic verrà istantaneamente creato:

```
show
Tutti i topic:
- Notizie
subscribe notiziario
Registrato come subscriber al topic 'notiziario'.
Comandi subscriber:
> listall
> quit
█
```

Mostriamo volontariamente che prima non era presente il topic “notiziario” e successivamente viene creato correttamente senza ulteriori istruzioni.

Ora il subscriber avrà la possibilità di effettuare il listAll per accedere a tutti i messaggi condivisi sul topic scelto. Esempio di un topic non vuoto:


```
listall
Tutti i messaggi sul topic 'Notizie':
- ID: msg_6
  Testo: 'Clamorosa alluvione'
  Data: 21/11/2024 - 16:25:18.00154
- ID: msg_5
  Testo: 'Clamorosa vittoria di Verstappen mette un'ipoteca sul campionato'
  Data: 21/11/2024 - 15:58:57.76364
- ID: msg_1
  Testo: 'L'italia non riesce a superare i francesi di Deschamps'
  Data: 21/11/2024 - 15:52:43.75897
- ID: msg_2
  Testo: 'Sinner vince le finali ATP di Torino'
  Data: 21/11/2024 - 15:55:21.42771
- ID: msg_3
  Testo: 'Juric viene esonerato dalla Roma, la societ? cerca un allenatore per sostituire l'allenatore croato'
  Data: 21/11/2024 - 15:56:10.92575
- ID: msg_4
  Testo: 'La Roma sceglie Ranieri come nuovo allenatore'
  Data: 21/11/2024 - 15:56:44.12476
```

Il subscriber avrà anche la possibilità di effettuare il quit, come il publisher. Presentiamo un recap della sessione del subscriber per fare il confronto con la versione del server.

```
PS C:\Users\Paolo\Documents\GitHub\LABSO_FIFONI\Project_FIFONI\Client> java Client localhost 9000
Connesso al server.
Comandi client:
> publish <topic>
> subscribe <topic>
> show
> quit
show
Tutti i topic:
- Notizie
- notiziario
subscribe Notizie
Registrato come subscriber al topic 'Notizie'.
Comandi subscriber:
> listall
> quit
listall
Tutti i messaggi sul topic 'Notizie':
- ID: msg_6
  Testo: 'Clamorosa alluvione'
  Data: 21/11/2024 - 16:25:18.00154
- ID: msg_5
  Testo: 'Clamorosa vittoria di Verstappen mette un'ipoteca sul campionato'
  Data: 21/11/2024 - 15:58:57.76364
- ID: msg_1
  Testo: 'L'italia non riesce a superare i francesi di Deschamps'
  Data: 21/11/2024 - 15:52:43.75897
- ID: msg_2
  Testo: 'Sinner vince le finali ATP di Torino'
  Data: 21/11/2024 - 15:55:21.42771
- ID: msg_3
  Testo: 'Juric viene esonerato dalla Roma, la societ? cerca un allenatore per sostituire l'allenatore croato'
  Data: 21/11/2024 - 15:56:10.92575
- ID: msg_4
  Testo: 'La Roma sceglie Ranieri come nuovo allenatore'
  Data: 21/11/2024 - 15:56:44.12476
quit
Sender terminato.
Connessione chiusa.
Receiver terminato.
Socket chiusa.
```

Riguardo alla sessione sopra, il server visualizzerà:

```
Client connesso.
Thread Thread[#30,Thread-9,5,main] in ascolto...
Richiesta: show
Richiesta: subscribe Notizie
Richiesta: listall
Richiesta: quit
Connessione chiusa.
```

Sessione interattiva e chiusura del Server

```
show
Tutti i topic:
- Notizie
- notiziario
inspect
ERRORE: nessun topic selezionato.
inspect Notizie

* SESSIONE INTERATTIVA AVVIATA
Comandi sessione interattiva:
> :listall
> :delete <id>
> :end
listall
Comando <listall> sconosciuto.
:listall
Tutti i messaggi sul topic 'Notizie':
- ID: msg_6
  Testo: 'Clamorosa alluvione'
  Data: 21/11/2024 - 16:25:18.00154
- ID: msg_5
  Testo: 'Clamorosa vittoria di Verstappen mette un'ipoteca sul campionato'
  Data: 21/11/2024 - 15:58:57.76364
- ID: msg_1
  Testo: 'L'Italia non riesce a superare i francesi di Deschamps'
  Data: 21/11/2024 - 15:52:43.75897
- ID: msg_2
  Testo: 'Sinner vince le finali ATP di Torino'
  Data: 21/11/2024 - 15:55:21.42771
- ID: msg_3
  Testo: 'Juric viene esonerato dalla Roma, la societ? cerca un allenatore per sostituire l'allenatore croato'
  Data: 21/11/2024 - 15:56:10.92575
- ID: msg_4
  Testo: 'La Roma sceglie Ranieri come nuovo allenatore'
  Data: 21/11/2024 - 15:56:44.12476
:delete msg_3
Messaggio 'msg_3' eliminato.
:listall
Tutti i messaggi sul topic 'Notizie':
- ID: msg_6
  Testo: 'Clamorosa alluvione'
  Data: 21/11/2024 - 16:25:18.00154
- ID: msg_5
  Testo: 'Clamorosa vittoria di Verstappen mette un'ipoteca sul campionato'
  Data: 21/11/2024 - 15:58:57.76364
- ID: msg_1
  Testo: 'L'Italia non riesce a superare i francesi di Deschamps'
  Data: 21/11/2024 - 15:52:43.75897
- ID: msg_2
  Testo: 'Sinner vince le finali ATP di Torino'
  Data: 21/11/2024 - 15:55:21.42771
- ID: msg_4
  Testo: 'La Roma sceglie Ranieri come nuovo allenatore'
  Data: 21/11/2024 - 15:56:44.12476
```

Da questo esempio vediamo come attivare correttamente una “sessione interattiva” del server. Prima di tutto possiamo usare il comando “show” per visualizzare i topic creati (1), successivamente possiamo sfruttare il comando inspect per effettuare una ispezione sul topic desiderato. Attenzione, nel caso di inspect senza il topic (2), verrà visualizzato “ERRORE: nessun topic selezionato”. Usiamo ora l’inspect correttamente (3). Inizia la “sessione interattiva” (4). Sfruttiamo il “listAll” simile ai comandi client. In questo caso dobbiamo usare i due punti altrimenti viene visualizzato l’errore (5) “Comando <listAll> sconosciuto”. Con il “listAll” corretto (6), vengono mostrati tutti i messaggi inviati sul topic in questione. Successivamente possiamo selezionare l’ID di un messaggio ed eliminarlo dal topic (7). Se effettuiamo il “listAll” una seconda volta, vedremo che manca il “msg_3”.

Nel caso in cui l’ID inserito non sia corretto verrà visualizzato il messaggio:

```
:delete messaggio5
ERRORE: messageID 'messaggio5' inesistente.
```

Sempre lato server, quando chiudiamo il server in modo corretto verrà visualizzato il seguente messaggio:

```
quit
Interrompendo il server...
Interrompendo i vari children...
Interrompendo Thread[#22,Thread-1,5,...]
Interrompendo Thread[#23,Thread-2,5,...]
Interrompendo Thread[#24,Thread-3,5,...]
Interrompendo Thread[#25,Thread-4,5,...]
Interrompendo Thread[#26,Thread-5,5,...]
Interrompendo Thread[#27,Thread-6,5,...]
Interrompendo Thread[#28,Thread-7,5,...]
Interrompendo Thread[#29,Thread-8,5,...]
Interrompendo Thread[#30,Thread-9,5,...]
Interrompendo Thread[#31,Thread-10,5,...]
Thread principale terminato.
```

Nel precedente caso di sessione interattiva, il Client rimane in attesa di comandi dall'utente e non appena viene inviato un messaggio (diverso da quit) il Client visualizza:

```
show
quit
Receiver terminato.
█
```

Successivamente inviando un qualsiasi messaggio, operazione necessaria per chiudere il `nextLine` che attende l'inserimento di una stringa, verrà visualizzato:

```
show
quit
Receiver terminato.
quit
Sender terminato.
Socket chiusa.
```

Nel caso in cui il server venga chiuso incorrettamente, sul client viene visualizzato:

```
Receiver terminato.
```

Se poi proviamo ad usare qualsiasi messaggio si visualizzerà:

```
Receiver terminato.
show
Sender terminato.
Socket chiusa.
```